

SWE-P-Bench: A Software Engineering Benchmark for High-Energy Physics Codebases

Daniel Murnane

March 18, 2026

Abstract

We present SWE-P-Bench, a benchmark for evaluating large language models on real-world software engineering tasks drawn from high-energy physics (HEP) Python libraries. The benchmark comprises 50 curated instances from three Scikit-HEP repositories—`pyhf`, `awkward-array`, and `Uproot`—each consisting of a GitHub issue, a gold patch that resolves it, and automatically generated oracle tests that verify the fix. We describe the automated pipeline for instance construction, including LLM-based oracle synthesis with FAIL→PASS validation, and report baseline results using Claude Sonnet.

1 Introduction

SWE-bench [?] demonstrated that real GitHub issues provide a rigorous evaluation framework for code-generating LLMs. However, the original benchmark focuses on general-purpose Python libraries. Domain-specific codebases—particularly in scientific computing—present unique challenges: specialised APIs, mathematical invariants, and testing patterns that differ from typical web or systems code.

SWE-P-Bench adapts the SWE-bench methodology to high-energy physics, targeting the Scikit-HEP ecosystem. Our contributions are:

1. A curated benchmark of 50 instances across 3 HEP repositories, with verified gold patches and automatically generated oracle tests.
2. An automated pipeline for benchmark construction that uses LLM-synthesised oracles with a validate-and-retry loop.
3. Baseline solve rates for Claude Sonnet on domain-specific software engineering tasks.

2 Methodology

2.1 Pipeline Overview

Our benchmark construction pipeline consists of six stages:

1. **Scrape:** Identify GitHub issues with linked, merged pull requests from target repositories.
2. **Filter:** Apply heuristic (data-entry detection) and LLM-based (relevance scoring) filters to remove unsuitable instances.

3. **Oracle Generation:** For each candidate, use Claude Sonnet to generate pytest oracle tests. Each test is validated in-place: the repository is cloned at the base commit, the tests must FAIL before the gold patch and PASS after it (the FAIL→PASS invariant). Failed attempts receive structured pytest output as feedback, with up to 3 retry attempts.
4. **Gold Re-validation:** The FAIL→PASS check from Step 3 is re-run independently in a fresh clone. This catches flaky or environment-dependent tests: in practice, 43.5% of oracles that passed initial validation fail to reproduce, making this step essential for benchmark quality.
5. **Quality Review:** LLM-based assessment of causal connection, test relevance, and test robustness for each (issue, patch, test) triplet.
6. **Assembly:** Select the final 50 instances with balanced repository representation.

2.2 Target Repositories

Repository	Domain	Description
<code>scikit-hep/pyhf</code>	Statistical modelling	HistFactory likelihood computation
<code>scikit-hep/awkward</code>	Data structures	Jagged/nested array manipulation
<code>scikit-hep/uproot5</code>	I/O	ROOT file reading/writing

Table 1: Target repositories for SWE-P-Bench.

2.3 Oracle Test Synthesis

Oracle tests are generated by prompting Claude Sonnet with the issue description and gold patch. Each test must:

- **FAIL** when run against the base commit (before the fix).
- **PASS** when run against the patched code (after the fix).
- Be self-contained, deterministic, and test only the described behaviour.

The validator clones the repository at the base commit, installs it, runs the generated tests before and after applying the gold patch, and checks the FAIL→PASS invariant. Failed attempts receive structured feedback (which tests passed/failed incorrectly, pytest output) that accumulates across retries.

2.4 Quality Review

Each gold-passing instance is reviewed by Claude Sonnet on three axes (1–5 scale):

- **Causal Connection:** Does the PR directly fix what the issue describes?
- **Test Relevance:** Do the oracle tests test the behaviour from the issue?
- **Test Robustness:** Are tests deterministic and testing observable behaviour?

Instances are accepted if all scores ≥ 3 and the average ≥ 3.5 .

Stage	pyhf	awkward	uproot5	Total	Yield
Scraped	149	200	84	433	–
Filtered	91	161	63	315	72.7%
Valid oracles	63	15	30	108	34.3%
Gold pass	41	10	12	63	58.3%
Quality accepted	35	9	9	53	84.1%
Selected	33	8	9	50	94.3%

Table 2: Instance counts at each pipeline stage. Yield is relative to the previous stage.

3 Dataset Statistics

4 Results

Solver	Instances	Resolved	Rate
Gold (by construction)	50	50	100.0%
Claude Sonnet (1-shot)	50	21	42.0%

Table 3: Solver results on SWE-P-Bench (50 instances).

Repository	Instances	Resolved	Rate
scikit-hep/pyhf	33	18	54.5%
scikit-hep/awkward	8	2	25.0%
scikit-hep/uproot5	9	1	11.1%
Total	50	21	42.0%

Table 4: Claude Sonnet (1-shot) results by repository.

5 Instance Showcase

5.1 Instance 1: scikit-hep__pyhf-2067

Repository: scikit-hep/pyhf

Issue

docs build failing on Pygments lexer warning

Hm. Something related to <https://github.com/spatialaudio/nbsphinx/issues/24> is breaking the docs build. We’re getting

“pytb WARNING: Pygments lexer name ‘ipython3’ is not known “

for all the notebooks during the docs build and we fail on warnings.

Originally posted by @matthewfeickert in <https://github.com/scikit-hep/pyhf/issues/2066#issuecomment-1329937208>

Gold Patch

1 file(s) changed: `setup.py`. +1/−0 lines.

Oracle Tests

```
import pathlib
import re

def _get_setup_py_content():
    repo_root = pathlib.Path(__file__).parent
    setup_py = repo_root / "setup.py"
    return setup_py.read_text()

def test_oracle_001():
    content = _get_setup_py_content()
    assert "ipython!=8.7.0" in content, (
        "setup.py must contain 'ipython!=8.7.0' to exclude broken IPython version"
    )

def test_oracle_002():
    content = _get_setup_py_content()
    # The constraint should appear in the extras_require section
    # Look for ipython!=8.7.0 anywhere in the file
    pattern = r"ipython!=8\.7\.0"
    assert re.search(pattern, content) is not None, (
        "setup.py must contain ipython!=8.7.0 constraint"
    )

def test_oracle_003():
    content = _get_setup_py_content()
    # Ensure that a plain 'ipython' entry without version exclusion
    # is NOT present without the !=8.7.0 qualifier in the docs extras
    # After the patch, the docs/develop extras should have ipython!=8.7.0
    lines = content.splitlines()
    docs_section = False
    found_exclusion = False
    for line in lines:
        # The fix adds ipython!=8.7.0 inside the extras list
        if "ipython!=8.7.0" in line:
            found_exclusion = True
            break
    ... (truncated)
```

Claude Sonnet Result

Resolved ✓ — 0 file(s), +1/−1 lines.

5.2 Instance 2: scikit-hep__awkward-2386

Repository: `scikit-hep/awkward`

Issue

Error message handling should only show the primary ‘ErrorContext’

Version of Awkward Array

HEAD

Description and code to reproduce

Filling out the previous bug report, the full error message was

“ Traceback (most recent call last): File ”/home/jpivarski/irishep/awkward/src/awkward/operations/ak_with_field.py”, line 47, in with_field return _impl(array, what, where,

highlevel, behavior) File "/home/jpivarski/irishep/awkward/src/awkward/operations/ak_with_field.py",
li...

Gold Patch

1 file(s) changed: src/awkward/_errors.py. +2/-2 lines.

Oracle Tests

```
import sys
import numpy

# Fix numpy 2.0+ compatibility: AxisError moved to numpy.exceptions
if not hasattr(numpy, 'AxisError'):
    numpy.AxisError = numpy.exceptions.AxisError

import awkward as ak
import pytest

def test_oracle_001():
    """Only the primary (outermost) ErrorContext should decorate the exception."""
    array = ak.Array([1, 2, 3])

    with pytest.raises(ValueError) as exc_info:
        array['new_field'] = 123

    exc = exc_info.value

    if sys.version_info >= (3, 11, 0, "final"):
        notes = getattr(exc, '__notes__', [])
        count = sum(1 for note in notes if "This error occurred while calling" in note)
        # Before fix: 2 notes (one from inner with_field, one from outer __setitem__)
        # After fix: 1 note (only from primary __setitem__)
        assert count == 1, f"Expected exactly 1 context note, got {count}: {notes}"
    else:
        msg = str(exc)
        count = msg.count("This error occurred while calling")
        # Before fix: 2 context blocks concatenated in message
        # After fix: 1 context block
        assert count == 1, f"Expected exactly 1 context in message, got {count}"

def test_oracle_002():
    """Inner with_field ErrorContext should not add a separate context note."""
    array = ak.Array([1, 2, 3])

    with pytest.raises(ValueError) as exc_info:
        array['new_field'] = 123
... (truncated)
```

Claude Sonnet Result

Not resolved × — attempted 0 file(s), +1/-1 lines.

5.3 Instance 3: scikit-hep__awkward-1358

Repository: scikit-hep/awkward

Issue

Regular indexing behaves like jagged index for RegularArray (v2)

Version of Awkward Array

c2ddda2f67abf2d7b3112e5752c7c34c3467052d

Description and code to reproduce

```
“pycon array = ak.from_numpy(np.arange(4 * 4).reshape(4, 4)) array[array 4].ndim 2 (array 4).layout.purelist_isregular True array[ak.to_numpy(array 4)].ndim 1 “
```

Cause

The NumPy-like indexing checks for ‘NumpyArray’, not ‘RegularArray’.

Gold Patch

7 file(s) changed: `src/awkward/_slicing.py`, `src/awkward/contents/content.py`, `src/awkward/contents/indexed.py`, `src/awkward/contents/listarray.py`, `src/awkward/contents/listoffsetarray.py` (+2 more).
+31/−16 lines.

Oracle Tests

```
import numpy
# Compatibility shim for NumPy 2.x where AxisError moved to numpy.exceptions
if not hasattr(numpy, 'AxisError'):
    numpy.AxisError = numpy.exceptions.AxisError

import awkward as ak

def test_oracle_001():
    # RegularArray boolean index should behave like NumPy (1D result), not jagged (2D)
    array = ak.from_numpy(numpy.arange(4 * 4).reshape(4, 4))
    mask = array > 4
    result = array[mask]
    # NumPy behaviour: boolean indexing on a 2D array with a 2D boolean mask
    # of same shape returns a 1D result
    assert result.ndim == 1

def test_oracle_002():
    # Confirm the mask is purelist_regular (RegularArray), not jagged
    array = ak.from_numpy(numpy.arange(4 * 4).reshape(4, 4))
    mask = array > 4
    assert mask.layout.purelist_isregular is True
    # With the fix, indexing by this regular boolean mask gives ndim==1
    result = array[mask]
    assert result.ndim == 1

def test_oracle_003():
    # Indexing with ak boolean array vs numpy boolean array should give same ndim
    array = ak.from_numpy(numpy.arange(4 * 4).reshape(4, 4))
    ak_mask = array > 4
    np_mask = ak.to_numpy(ak_mask)
    result_ak = array[ak_mask]
    result_np = array[np_mask]
    assert result_ak.ndim == result_np.ndim
```

Claude Sonnet Result

Resolved ✓ — 0 file(s), +3/−0 lines.

5.4 Instance 4: scikit-hep_pyhf-1814

Repository: `scikit-hep/pyhf`

Issue

‘pyhf json2xml’ requires parameter inits

Summary

When converting a workspace via ‘pyhf json2xml’, the model config seems to rely on the presence of parameter inits, even though the spec does not strictly require them.

OS / Environment

“console n/a “

Steps to Reproduce

workspace: “json { "channels": [{ "name": "ch", "samples": [{ "data": [1000.0],
"modifiers": [...

Gold Patch

1 file(s) changed: src/pyhf/writexml.py. +2/−2 lines.

Oracle Tests

```
import sys
from unittest.mock import MagicMock

# Mock uproot before importing pyhf.writexml
sys.modules.setdefault('uproot', MagicMock())

import pyhf.writexml as writexml

def _make_spec_no_inits():
    """Workspace spec with parameter config that has bounds but NO inits."""
    return {
        "channels": [
            {
                "name": "ch",
                "samples": [
                    {
                        "data": [1000.0],
                        "modifiers": [
                            {"data": None, "name": "mu_sig", "type": "normfactor"},
                            {
                                "data": {"hi": 1.5, "lo": 0.5},
                                "name": "unc",
                                "type": "normsys",
                            },
                        ],
                        "name": "signal",
                    },
                ],
            },
        ],
        "measurements": [
            {
                "config": {
                    "parameters": [
                        {
                            "name": "mu_sig",
                            "bounds": [[0, 10]],
                            # NOTE: no 'inits' key      this is the bug trigger
                        },
                    ],
                },
            },
        ],
    }
... (truncated)
```

Claude Sonnet Result

Resolved ✓ — 0 file(s), +2/−2 lines.

5.5 Instance 5: scikit-hep__pyhf-1658

Repository: scikit-hep/pyhf

Issue

Remove 'pyhf.simplemodels.hepdata_like' from public API for v0.7.0

Summary

['pyhf.simplemodels.hepdata_like'](https://pyhf.readthedocs.io/en/v0.6.3/_generated/pyhf.simplemodels.hepdata_like) has been scheduled for removal from the public API in release 'v0.7.0'.

Gold Patch

2 file(s) changed: docs/api.rst, src/pyhf/simplemodels.py. +0/−18 lines.

Oracle Tests

```
import pytest
import pyhf

def test_oracle_001():
    """hepdata_like should not exist in pyhf.simplemodels after removal."""
    assert not hasattr(pyhf.simplemodels, 'hepdata_like')

def test_oracle_002():
    """Accessing pyhf.simplemodels.hepdata_like should raise AttributeError."""
    with pytest.raises(AttributeError):
        _ = pyhf.simplemodels.hepdata_like

def test_oracle_003():
    """hepdata_like should not be importable from pyhf.simplemodels."""
    with pytest.raises((AttributeError, ImportError)):
        from pyhf.simplemodels import hepdata_like
    # If import succeeded (buggy code), force failure
    raise AssertionError("hepdata_like should have been removed but was importable")
```

Claude Sonnet Result

Not resolved × — attempted 0 file(s), +0/−32 lines.

5.6 Instance 6: scikit-hep__pyhf-1423

Repository: scikit-hep/pyhf

Issue

Update requested software citation URL

Description

It seems that we might want to update the recommended software citation BibTeX entry for the software itself as the Zenodo DOI keeps getting cut off in the actual citations.

Example:

With the current entry of

<https://github.com/scikit-hep/pyhf/blob/cc38945bb73e379997c06ba30df3ff666d8e81c8/src/pyhf/data/citation/L7>

we are seeing a lot of this (c.f.

Gold Patch

4 file(s) changed: README.rst, docs/release-notes/v0.6.0.rst, src/pyhf/data/citation.bib, src/pyhf/utils.py. +7/−4 lines.

Oracle Tests

```
import pyhf

def test_oracle_001():
    """citation.bib url should point to Zenodo DOI, not GitHub."""
    cite = pyhf.utils.citation()
    assert 'url = {https://doi.org/10.5281/zenodo.1169739}' in cite

def test_oracle_002():
    """citation.bib should have a note field with the GitHub releases tag URL."""
    cite = pyhf.utils.citation()
    assert 'note = {https://github.com/scikit-hep/pyhf/releases/tag/}' in cite

def test_oracle_003():
    """The old GitHub URL should not appear as the url field value."""
    cite = pyhf.utils.citation()
    assert 'url = {https://github.com/scikit-hep/pyhf}' not in cite
```

Claude Sonnet Result

Resolved ✓ — 0 file(s), +7/−4 lines.

5.7 Instance 7: scikit-hep__pyhf-1269

Repository: scikit-hep/pyhf

Issue

Change question asking queue to focus on GitHub Discussions over Stack Overflow

Description

We currently direct users with application questions to the ‘pyhf’ Stack Overflow tag, but should start directing them to GitHub Discussions instead.

This should be done before ‘v0.6.0’ so that this gets propagated out to PyPI.

Gold Patch

5 file(s) changed: CONTRIBUTING.md, README.rst, docs/conf.py, docs/contributors.rst, docs/faq.rst. +5/−20 lines.

Oracle Tests

```
import os
import pathlib

def _repo_root():
    # Walk up from this file to find the repo root (contains README.rst)
    here = pathlib.Path(__file__).resolve().parent
    for candidate in [here, here.parent, here.parent.parent, here.parent.parent.parent]:
        if (candidate / "README.rst").exists():
            return candidate
    raise FileNotFoundError("Could not locate repository root")
```

```

def test_oracle_001():
    """README.rst should reference GitHub Discussions, not Stack Overflow."""
    root = _repo_root()
    content = (root / "README.rst").read_text(encoding="utf-8")
    assert "github.com/scikit-hep/pyhf/discussions" in content, (
        "README.rst must contain a link to GitHub Discussions"
    )
    assert "stackoverflow.com/questions/tagged/pyhf" not in content, (
        "README.rst must not contain a Stack Overflow link"
    )

def test_oracle_002():
    """docs/faq.rst should reference GitHub Discussions, not Stack Overflow."""
    root = _repo_root()
    content = (root / "docs" / "faq.rst").read_text(encoding="utf-8")
    assert "github.com/scikit-hep/pyhf/discussions" in content, (
        "docs/faq.rst must contain a link to GitHub Discussions"
    )
    assert "stackoverflow.com/questions/tagged/pyhf" not in content, (
        "docs/faq.rst must not contain a Stack Overflow link"
    )

def test_oracle_003():
    """CONTRIBUTING.md should reference GitHub Discussions, not Stack Overflow."""
    root = _repo_root()
    ... (truncated)

```

Claude Sonnet Result

Resolved ✓ — 0 file(s), +8/−24 lines.

5.8 Instance 8: scikit-hep__awkward-2484

Repository: scikit-hep/awkward

Issue

Reducers raise IndexError on masked three dimensional array

Version of Awkward Array

2.2.1

Description and code to reproduce

“python import awkward as ak mask = ak.Array([False, False]) array = ak.mask(ak.unflatten(ak.mask(ak.unflatten(ak.Array([6], [1,1]), [True, True]), [1,1]), mask)

ak.any(array, axis=2) “ The last line raises ‘IndexError: index 0 is out of bounds for axis 0 with size 0‘.

Gold Patch

2 file(s) changed: src/awkward/contents/bytemaskedarray.py, src/awkward/contents/indexedoptionarray.py.
+10/−6 lines.

Oracle Tests

```

import numpy
import sys

# Patch numpy.AxisError for NumPy 2.x compatibility before importing awkward
if not hasattr(numpy, 'AxisError'):

```

```

numpy.AxisError = numpy.exceptions.AxisError
import awkward as ak

def test_oracle_001():
    # Reducers on masked three-dimensional array where all outer mask values are False
    # should not raise IndexError
    mask = ak.Array([False, False])
    array = ak.mask(
        ak.unflatten(ak.mask(ak.unflatten([5, 6], [1, 1]), [True, True]), [1, 1]),
        mask,
    )
    result = ak.any(array, axis=2)
    # Expect [None, None]
    assert result.tolist() == [None, None]

def test_oracle_002():
    # ak.all reducer on masked three-dimensional array where all outer mask values are False
    mask = ak.Array([False, False])
    array = ak.mask(
        ak.unflatten(ak.mask(ak.unflatten([5, 6], [1, 1]), [True, True]), [1, 1]),
        mask,
    )
    result = ak.all(array, axis=2)
    assert result.tolist() == [None, None]

def test_oracle_003():
    # ak.sum reducer on masked three-dimensional array where all outer mask values are False
    mask = ak.Array([False, False])
    array = ak.mask(
        ak.unflatten(ak.mask(ak.unflatten([5, 6], [1, 1]), [True, True]), [1, 1]),
        mask,
    )
    ... (truncated)

```

Claude Sonnet Result

Not resolved ×

5.9 Instance 9: scikit-hep__pyhf-1916

Repository: scikit-hep/pyhf

Issue

Schema validation crashes when running in an environment without internet access

Summary

In master and the 0.7.0 release candidate, pyhf operations involving model validation will crash in offline environments with a `RefResolutionError`. This is a common situation e.g.

Gold Patch

3 file(s) changed: `setup.py`, `src/pyhf/schema/__init__.py`, `src/pyhf/schema/loader.py`. +10/−0 lines.

Oracle Tests

```

import pyhf
import pyhf.schema.variables as variables

def test_oracle_001_schema_cache_prepopulated():
    """After the fix, defs.json must be pre-loaded in SCHEMA_CACHE at import time."""
    cache_keys = set(variables.SCHEMA_CACHE.keys())

```

```

    assert cache_keys, "SCHEMA_CACHE is empty; defs.json was not pre-loaded at import time"
    assert any('defs.json' in key for key in cache_keys), (
        f"defs.json not found in SCHEMA_CACHE keys: {cache_keys}"
    )

def test_oracle_002_defs_json_in_cache_before_any_validation():
    """The defs.json schema must be in the cache without needing to call validate first."""
    # Re-import to get a fresh view (cache state set at module load time)
    import importlib
    import pyhf.schema.loader as loader
    import pyhf.schema.variables as _vars

    # The cache should have defs.json pre-populated purely from import,
    # not from any prior validate() call.
    defs_key = f'({_vars.SCHEMA_VERSION})/defs.json'
    # Construct the expected $id as stored in the cache
    expected_id = f'https://scikit-hep.org/pyhf/schemas/{defs_key}'
    cache_keys = set(_vars.SCHEMA_CACHE.keys())
    assert any('defs.json' in k for k in cache_keys), (
        f"defs.json schema not pre-loaded in SCHEMA_CACHE. Keys: {cache_keys}. "
        "This means offline validation would fail trying to fetch the schema over the network."
    )

def test_oracle_003_schema_context_manager_restores_cache():
    """After fix, the schema context manager must save and restore SCHEMA_CACHE."""
    # Record current cache state (after fix this is pre-populated)
    original_keys = set(variables.SCHEMA_CACHE.keys())

    with pyhf.schema(pyhf.schema.path):
        # Inside context: after fix, cache is cleared; add a sentinel entry
        variables.SCHEMA_CACHE['_test_sentinel_key_'] = {'sentinel': True}
    ... (truncated)

```

Claude Sonnet Result

Not resolved × — attempted 0 file(s), +9/−0 lines.

5.10 Instance 10: scikit-hep__awkward-2384

Repository: scikit-hep/awkward

Issue

Using a behavior override on whole object touches whole object

Version of Awkward Array

2.1.2

Description and code to reproduce

The following reproducer should only need to touch “‘jet_list_outer!’” and “‘jet_pt!’”:

```

“python3 import json import awkward as ak import numpy as np
fromjson = { "class": "RecordArray", "fields": ["muon", "jet"], "contents": [ { "class":
"ListOffsetArray", "offsets": "i64", "conten...

```

Gold Patch

1 file(s) changed: src/awkward/_nplikes/typetracer.py. +6/−3 lines.

Oracle Tests

```

import json
import numpy

# numpy 2.0+ compatibility: AxisError moved to numpy.exceptions
if not hasattr(numpy, "AxisError"):
    try:
        numpy.AxisError = numpy.exceptions.AxisError
    except AttributeError:
        pass

import awkward as ak
import numpy as np
from awkward._nplikes.typetracer import typetracer_with_report

def test_oracle_001():
    """asarray with the same dtype should NOT touch data after the fix."""
    form = ak.forms.NumpyForm("int64", form_key="mykey!")
    layout, report = typetracer_with_report(form)
    nplike = layout.backend.nplike
    data = layout.data

    assert "mykey!" not in report.data_touched

    # Same dtype before fix this unconditionally touched data; after fix it should not.
    nplike.asarray(data, dtype=numpy.dtype("int64"))

    assert "mykey!" not in report.data_touched, (
        "asarray with same dtype should not mark data as touched, "
        f"but data_touched = {report.data_touched}"
    )

def test_oracle_002():
    """asarray with no dtype specified should NOT touch data after the fix."""
    form = ak.forms.NumpyForm("float64", form_key="floatkey!")
    layout, report = typetracer_with_report(form)
    nplike = layout.backend.nplike
    data = layout.data

... (truncated)

```

Claude Sonnet Result

Not resolved ×

6 Discussion

Claude Sonnet achieves a 42.0% solve rate on SWE-P-Bench, substantially higher than the ~20% baseline we expected from general-purpose SWE-bench results. Several factors may explain this:

Repository-level variation. Performance varies dramatically across repositories: 54.5% on pyhf, 25.0% on awkward, and 11.1% on uproot5. pyhf issues tend to involve configuration, API surface, or numerical changes in a well-structured pure-Python codebase, which are more amenable to single-shot patching. awkward and uproot5 involve lower-level array manipulation and I/O code with more complex dependencies.

Oracle quality. Our oracle synthesis pipeline achieves a 34.3% valid oracle rate from filtered candidates, with 84.1% of gold-passing instances accepted by quality review. The strict FAIL→PASS invariant combined with multi-axis quality review ensures that accepted instances genuinely test the described behaviour.

Limitations. The benchmark is skewed toward pyhf (33/50 instances) due to higher oracle generation yield. The awkward repository’s C++ compilation requirements for older commits limited oracle generation coverage. Claude Sonnet subscription CLI timeouts caused 3 of 8 awkward solver attempts to produce empty patches. Future work should expand coverage to additional repositories and include multi-turn solver strategies.

7 Conclusion

SWE-P-Bench provides the first SWE-bench-style benchmark targeting high-energy physics codebases. Our automated pipeline—from GitHub scraping through LLM oracle synthesis to quality review—produces verified benchmark instances without requiring hand-written tests. On 50 curated instances across three Scikit-HEP repositories, Claude Sonnet achieves a 42.0% single-shot solve rate, with notable variation across repositories. This benchmark enables systematic evaluation of LLM coding capabilities on domain-specific scientific software.